CS6600_18_PA2

Deepak Charan S, Agnideb Mukherjee, Sooryanarayanan G

1 Introduction

This Assignment extends the Assignment-1 processor with seven independent execution pipelines and a pipelined, non-blocking Harvard cache hierarchy. The Cache also works with Tree-PLRU/ SRRIP replacement policy.

2 TODO A2-DESIGN:

2.1 TODO(A2-DESIGN-01):

- Created the top-level processor design in `BuildDesign()` using `ProcessorBuilder`.
- First constructed the five pipeline stages: `IF`, `ID`, `EX`, `MEM`, and `WB`.
- Then added architectural and intermediate pipeline-register nodes: `PC`, `IF/ID`, `ID/EX`, `EX/MEM`, and `MEM/WB`.
- Connected stage outputs to pipeline registers (using `kClockedData`), and each pipeline register to its following stage (using `kData`).
- `ID` and `MEM` are connected to instruction and data caches respectively.
- Connected the Decode stage to all seven execution units.
- Connected arithmetic and control execution results back to the Execute stage (the AGU result, however, is connected directly to the Memory stage because it produces effective addresses for load/store operations).
- Added `MainControl` and `HazardControl`. Main control feeds Decode, while hazard control can stall `PC` and `IF/ID` and inject a bubble into `ID/EX`.
- Added a CONTROL redirect connection to `PC`, allowing taken branches and jumps to replace the sequential next program counter.
- All these design choices were cross-verified with *builder_validation.cc*.

2.2 TODO(A2-DESIGN-02):

- Completing fetch stage.
- Checks whether fetch has been stopped, in which case updates the pc and next if-id with appropriate instruction and exits.
- If fetch is not stopped, then asks the machine for the next instruction at pc_ using `machine_->TryIssueFetch()`.
- Now, if the decode stage consumed the earlier instruction or the if-id register is free, it tries to put a new instruction into it.
- Checks whether the new instruction is available. Once it becomes available, updates the next if-id register.
- If the stage is stalled, sets the next pc and if-id to the current one.

2.3 TODO(A2-DESIGN-03):

- Completing decode stage, hazard checking for RAW and WAW, and Unit-For function.
- Checks whether the if-id register contains a valid instruction.
- Decode stage checks for these in the given order.
  - Completion Queue Capacity
  - RAW, WAW hazards
  - Execution unit availability
  - Completion slot reservation.
- Only if all the checks pass, the instruction is dispatched.
- Read the operands for lhs, rhs, and store value.
- Dispatch the instruction to the correct execution pipeline and removes it from the IF-ID.
- Every pipeline advances once per cycle, even if it isn't receiving a new instruction.

2.4 TODO(A2-DESIGN-05 EXECUTE):

- Completing the exec stage.
- Checks each object inside `pipelines_` for a valid result.
- Does not check AGU because it's handled separately by the Memory Stage.

- For all other instruction types, the result is sent by `CompleteNonMemory` and setting `consume_output` flag to true.
- Upon seeing a control redirect, updates the pc and invalidates wrong path fetches done.
- On a control halt, stops fetching, and for other control types, execution continues with the sequential pc.
- Multiple execution units can produce results in the same cycle, but retirement is still one instruction per cycle.

2.5 TODO(A2-DESIGN-05 MEMORY):

- Completing the mem stage.
- Checks whether the output of AGU has any value.
- On finding a valid result, sends a memory request to the data cache/memory subsystem for issue data.
- Read request for load, and write request for store of the store_value.
- If `machine_->TryIssueData()` returns true, consume the output; else, retry in the next cycle.
- Back-pressure between AGU and memory system.

2.6 TODO(A2-DESIGN-06):

- Completion of the write-back stage.
- Checks the retirement queue for the oldest instruction that is ready to retire.
- Checks if there is any instruction that can retire. This is done using `machine_->PeekRetirement()`.
- If there are no instructions to retire, write-back doesn't do anything.
- If there is an instruction, then the call `machine_->ApplyRetirement()` for actual commit.
- In order retirement.
- At most one instruction retires per cycle.

3 TODO A2-EXEC:

3.1 TODO(A2-EXEC-01):

- Added allocation of `latency + 1` empty stages for both the `current` and `next` execution-pipeline vectors.
- Added `Reset()` logic to clear all stage entries, reset the next legal dispatch boundaries to 0, and clear execution lifecycle flags.
- Added `BeginCycle()` logic to copy committed state from `current` into `next`, copy the current dispatch limit into its planned version, and prepare the unit for one `Operate()` call.
- Added relevant checks to detect malformed operations.

3.2 TODO(A2-EXEC-02):

- Added logic for all `ExecutionUnitKind` entries.
- Also completed `OutputIntent()` to show the value of the last stage in the pipeline.

3.3 TODO(A2-EXEC-03):

- Completed `CanAccept()` to determine if pipeline can accept incoming operation.

3.4 TODO(A2-EXEC-04):

- Completed `Operate()`, planning the pipeline for the next cycle.
  - Added validation checks to ensure that the pipeline has begun a cycle and has not already operated during that cycle. Also validated a new dispatch operation by checking that its opcode belongs to the execution unit and that its dispatch boundary satisfies the initiation-interval requirement.
  - Checked arithmetic overflow when calculating dispatch-related boundaries.
  - Retired the current final-stage output from the planned pipeline state when the CPU grants output consumption (And made the last stage empty.
  - Moved in-flight operations one stage forward (if the next stage was empty at the moment).
  - Inserted a valid dispatched operation into stage zero only when the planned stage was available.

  - Updated the planned next legal dispatch boundary after accepting a new operation.
  - Marked the execution pipeline as operated (enforcing at most one planned pipeline update per cycle).

3.5 TODO(A2-EXEC-05):

- Completed `Commit()`, ensuring all the state values are set properly and assigning `current` to the planned `next`
- Completed the `Snapshot()` function, capturing all the details of the current cycle.
- Wrote `HasWork()` to check for transient operations present in the pipeline.

4 TODO A2-CACHE:

4.1 TODO(A2-CACHE-01):

- The cache controller maintains bounded cache-line, MSHR, writeback, and pipeline state using separate current and next-state structures.
- Reset initializes the cache lines, pipeline entries, MSHRs, writeback entries, statistics, and replacement-policy state.

4.2 TODO(A2-CACHE-02):

- Implemented cache address mapping using the configured line size and number of sets.
- Each address is decomposed into its block address, set index, tag, and byte offset within the cache line.

4.3 TODO(A2-CACHE-03):

- Added validation for incoming cache requests.
- Requests are checked for legal request types, cache-line boundaries, alignment, size, origin information, and generation information depending on the cache level.

4.4 TODO(A2-CACHE-04):

- Implemented upward response arbitration between the current OUTPUT entry, responding MSHRs, and writeback acknowledgement waiters.
- The response is selected deterministically according to `(arrival_order, response_id)`.

- Implemented `ResponseIntent()` to expose the selected response without modifying cache state.

4.5 TODO(A2-CACHE-05):

- Implemented the check for whether a granted upper response drains the current OUTPUT stage, allowing same-cycle OUTPUT-space planning.
- The remaining three-stage backpressure logic is handled through the cache stage-intent mechanism.

4.6 TODO(A2-CACHE-06):

- Implemented lower-request arbitration between demand MSHRs and cache writebacks.
- Demand requests that overlap incomplete writebacks are prevented from being issued, while younger non-conflicting requests may proceed.
- Added the writeback bypass mechanism so that a writeback is forced after reaching the specified demand-bypass limit.
- Implemented `LowerRequestIntent()` to expose the selected lower request without modifying state.

4.7 TODO(A2-CACHE-07):

- Implemented cache-hit processing using the set and hit way obtained during lookup.
- For write and writeback hits, the corresponding bytes in the resident cache line are updated and the line is marked dirty.
- Constructed the appropriate read-data, write-acknowledgement, or writeback-acknowledgement response.
- Updated replacement-policy state on a hit and maintained the corresponding hit statistics.

4.8 TODO(A2-CACHE-08):

- Added deterministic victim selection, preferring an eligible invalid cache way before invoking the configured replacement policy.
- Constructed MSHR waiters containing the request identity, ordering, address, size, and payload information required for later replay.
- Implemented merging of an additional request into an already existing MSHR.

- The merged request is appended as a waiter, the MSHR-merge counter is updated exactly once, and the LOOKUP entry is consumed.

4.9 TODO(A2-CACHE-09):

- This part handles installation of a returned cache-line fill and ordered replay of the requests waiting on the corresponding MSHR.

4.10 TODO(A2-CACHE-10):

- Added construction of writeback acknowledgement waiters containing the information required to reconstruct the upstream acknowledgement.
- Upper-level writebacks can be forwarded toward the lower memory hierarchy through the writeback-buffer path.

4.11 TODO(A2-CACHE-11):

- This part manages ordering and merging of overlapping writeback entries before they are issued to the lower level.

4.12 TODO(A2-CACHE-12):

- Implemented handling of granted upper responses.
- When an MSHR response waiter is granted, it is marked as sent. After all waiters have been sent, its reserved cache way is unpinned and the MSHR entry is released.
- Similarly, writeback acknowledgement waiters are marked as sent, and the writeback entry is released once all acknowledgement waiters have completed.
- Lower-request grants transition the selected MSHR or writeback request from `kUnsent` to `kInFlight` and maintain the writeback-bypass accounting.

4.13 TODO(A2-CACHE- 13):

- Added cache statistics accounting for the different lookup-stall conditions.
- The controller also provides snapshots of its pipeline entries, MSHRs, writebacks, cache lines, and replacement-policy state.
- `HasWork()` reports whether transient cache work remains in the pipeline, MSHRs, or writeback buffer.

5 TODO A2-REPL:

5.1 TODO(A2-REPL-01):

- Design **Tree-PLRU** cache replacement policy.
  - Set up proper state values in `Reset()`, `BeginCycle()`, `Commit()`.
  - `OnHit()` and `OnInsert()` both traverse the tree (using the pointers from the bits), updating the bits to point towards the less-recently used sections of the cache (Given set and way are first validated).
  - `SelectVictim()` and `PeekVictim()` also traverse the tree to find an eligible way to replace if eviction is needed (if the subtree mentioned by the Tree bits doesn't point to any eligible way, we take the alternative subtree).

5.2 TODO(A2-REPL-02):

- Design **SRRIP** cache replacement policy.
  - Set up proper state values in `Reset()`, `BeginCycle()`, `Commit()`.
  - `OnHit()` marks **RRPV** of the given way in the set as 0, while `OnInsert()` marks it as 2 (Given set and way are first validated).
  - `SelectVictim()` and `PeekVictim()` find the least recently used eligible way (lowest index if tie-breaker) and mark it for eviction if needed (other entries are appropriately aged if the marked entry doesn't have the highest possible **RRPV** yet).